

Universidade da Beira Interior

Departamento de Informática



**Departamento de
Informática**

Uma Abordagem Híbrida Quântico-Clássica ao Escalonamento de Processos via QAOA

Elaborado por:

Tomás Simões Duarte

Orientador:

Professor Doutor Paul Andrew Crocker

7 de julho de 2026

Conteúdo

Conteúdo	i
Lista de Figuras	iii
Lista de Tabelas	iv
Lista de Equações	v
1 Introdução	1
1.1 Motivação	1
1.2 Formulação do Problema	1
2 Formulação QUBO para Escalonamento	3
2.1 CPU Scheduling em Sistemas Operativos	3
2.2 Formulação QUBO	3
2.3 Construção da Matriz Q	4
2.4 Exemplo Ilustrativo	5
3 Resolução do QUBO com QAOA	7
3.1 Conversão do QUBO em Hamiltoniano	7
3.2 Mixer X e XY	9
3.2.1 Mixer X	9
3.2.2 Mixer XY	9
3.3 Circuito Variacional e Otimização Clássica	10
3.4 Medição, Descodificação e Seleção da Solução	11
4 Decomposição Adaptativa	12
4.0.1 Verificações de Qualidade e Divisão Recursiva	14
4.0.2 Exemplo da Formação de Bundles	15
5 Decomposição em Sub-QUBOs	17
5.1 Construção do QUBO Global	17
5.2 Extração de um Sub-QUBO	17
5.3 Resolução Iterativa	18
5.3.1 Exemplo de Extração dos Sub-QUBOs	19
5.3.1.1 Primeiro sub-QUBO	19
5.3.1.2 Segundo sub-QUBO	20
6 Arquitetura e Implementação	21
6.1 Da Visão Geral à Implementação	22
6.2 Estrutura do Projeto	26
6.2.1 Interfaces de Dados	26
6.2.2 Configurações	27
6.3 Conclusão	27

7	Investigação	28
7.1	Arquitetura dos Experimentos	28
7.1.1	Definição e Execução dos Cenários	29
7.1.2	Recolha e Armazenamento dos Resultados	30
7.2	Resultados da Investigação	31
7.2.1	Validação Inicial da Pipeline	31
7.2.1.1	Validação da Decomposição	31
7.2.1.2	Caracterização Inicial do QAOA	32
7.3	Escolha do Mixer	34
7.3.1	Viabilidade sob orçamento reduzido	34
7.3.2	Comparação com Valores Práticos de <code>top_k</code>	35
7.3.3	Síntese	35
7.4	Limites da Pipeline	36
7.4.1	Instâncias com Processo Dominante	36
7.4.2	Instâncias com Pesos Quase Iguais	36
7.4.3	Instâncias com Estrutura CPU/I/O	37
8	Conclusões e Trabalho Futuro	38
8.1	Conclusões Principais	38
8.2	Trabalho Futuro	38
	Bibliografia	39

Lista de Figuras

4.1	Representação simplificada do grafo de afinidade. Algumas ligações foram omitidas para facilitar a leitura.	16
6.1	Visão geral do fluxo da arquitetura.	21
6.2	Interfaces de dados entre aquisição, SystemSnapshot e Workload.	23
6.3	Fluxo final de construção, resolução e validação das QUBOInstance.	25
7.1	Arquitetura utilizada na execução dos experimentos da investigação.	29

Lista de Tabelas

4.1	Processos usados no exemplo de formação de bundles.	15
4.2	Bundles e respectivas grandezas agregadas.	16
5.1	Bias introduzido no segundo sub-QUBO pela carga acumulada.	20
5.2	Resultado final do exemplo de resolução por sub-QUBOs.	20
6.1	Mapa resumido da implementação.	26
6.2	Interfaces de dados centrais.	26
6.3	Configurações principais da implementação.	27
7.1	Resultados dos testes iniciais em instâncias pequenas.	31
7.2	Escalabilidade da pipeline com o número de processos.	31
7.3	Comparação entre execução iterativa e direta para diferentes limites de qubits ($N = 8, K = 2$).	32
7.4	Impacto da profundidade e do número de iterações na resolução direta com <code>qubit_max=16</code> ($N = 8, K = 2$).	33
7.5	Viabilidade e otimalidade com <code>top_k=1</code> e $p = 1$	34
7.6	Otimalidade por mixer, profundidade e <code>top_k</code> com 100 passos e P_{safe}	35
7.7	Resultados com processo dominante ($N = 6, K = 2$).	36
7.8	Resultados com pesos quase iguais ($N = 4, K = 2$).	37
7.9	Resultados do clustering CPU/I/O para diferentes valores de <code>io_alpha</code> ($N = 6, K = 2$).	37

Lista de Equações

2.1	Energia QUBO	3
2.2	Problema de otimização QUBO	3
2.3	Desvio quadrático total das cargas	4
2.4	Carga média alvo	4
2.5	Carga total no núcleo j	4
2.6	Expansão do termo de balanceamento	5
2.7	Coeficiente diagonal do balanceamento	5
2.8	Equivalência entre o desvio de carga e a energia QUBO	5
2.9	Penalidade de Atribuição Única	5
2.10	Coeficientes da matriz Q	5
2.11	Indexação linear das variáveis no exemplo	6
2.12	Termos diagonais do balanceamento no exemplo	6
2.13	Matriz de balanceamento do exemplo	6
2.14	Matriz Q final do exemplo	6
2.15	Comparação de energia entre duas soluções viáveis	6
3.1	Energia QUBO a converter em Hamiltoniano	7
3.3	Mapeamento de variável binária para operador de Pauli Z	7
3.4	Conversão de um termo linear do QUBO	8
3.5	Conversão de um termo quadrático do QUBO	8
3.6	Hamiltoniano de custo	8
3.7	Energia associada a uma bitstring	8
3.8	Unitária de custo do QAOA	8
3.9	Fase induzida pela energia da bitstring	8
3.10	Unitária de mistura do QAOA	8
3.11	Hamiltoniano do mixer X	9
3.12	Estado inicial uniforme do mixer X	9
3.13	Estados possíveis para um processo e dois núcleos	9
3.14	Estados atribuição única válidos	9
3.15	Bloco de qubits associado a um processo	10
3.16	Interação do mixer XY	10
3.17	Troca de amplitude entre atribuições	10
3.18	Dimensão do espaço binário total	10
3.19	Número de atribuições atribuição única válidas	10
3.20	Circuito QAOA com p camadas	10
3.21	Otimização dos parâmetros variacionais	10
3.22	Energia clássica usada na seleção final	11
4.1	Número de qubits necessários sem decomposição	12
4.2	Número máximo de processos por bundle	12
4.3	Número adaptativo de bundles	13
4.4	Carga efetiva de CPU	13
4.5	Vetor de características de um processo	13
4.6	Distâncias quadráticas nas componentes CPU e memória	13
4.7	Afinidades RBF para CPU e memória	13

4.8	Matriz de afinidade combinada	14
4.9	Atribuição de processos a bundles	14
4.10	Carga de CPU e memória agregadas por bundle	14
4.11	Coefficiente de variação de um bundle	14
4.12	Z-score da carga agregada de um bundle	14
5.1	Conjunto de entidades após a decomposição adaptativa	17
5.2	Variável de atribuição entidade-núcleo na decomposição iterativa . .	17
5.3	Partição das entidades em grupos	17
5.4	Número máximo de entidades por sub-QUBO	18
5.5	Carga acumulada por núcleo antes da iteração t	18
5.6	Propagação da carga acumulada como bias diagonal	18
5.7	Balanceamento local com carga acumulada	18
5.8	Atualização da carga acumulada após resolver um sub-QUBO	18

Acrónimos

CFS *Completely Fair Scheduler*

CPU *Central Processing Unit*

GPU *Graphics Processing Unit*

IO *Input/Output*

NISQ *Noisy Intermediate-Scale Quantum*

QAOA *Quantum Approximate Optimization Algorithm*

QUBO *Quadratic Unconstrained Binary Optimization*

RSS *Resident Set Size*

Introdução

Os sistemas operativos modernos distribuem carga entre núcleos de *Central Processing Unit* (CPU) recorrendo a métodos clássicos [1] que, apesar de eficazes em produção, tomam decisões subótimas em cenários de elevada contenção de recursos.

Em paralelo, métodos de otimização quântica, como o *Quantum Approximate Optimization Algorithm* (QAOA) [2], têm sido investigados em problemas combinatoriais que podem ser formulados como *Quadratic Unconstrained Binary Optimization* (QUBO) [3, 4].

Este relatório apresenta uma plataforma híbrida quântico-clássica para testar o problema de atribuição de processos a núcleos de CPU. A plataforma isola a decisão de atribuição processo-núcleo, modela-a como um QUBO, resolve-a com QAOA via PennyLane [5], e valida os resultados por comparação com um *solver* exato de *brute force*. O objetivo não é demonstrar superioridade quântica face a heurísticas clássicas, mas sim caracterizar o comportamento do QAOA neste domínio.

1.1 Motivação

A motivação para este projeto surgiu do interesse em compreender de que forma a computação quântica pode ser aplicada a problemas práticos da computação clássica. Uma primeira intuição levou a considerar a possibilidade de um sistema operativo assistido por computação quântica — ideia que, após análise, se revelou demasiado ampla para o estado atual do hardware quântico. No entanto, no mesmo âmbito, o escalonamento de processos surgiu como um subproblema familiar: trata-se de um problema de otimização combinatória onde, para um conjunto de processos e núcleos, se procura a distribuição de carga mais equilibrada entre múltiplas configurações possíveis. Esta estrutura mapeia-se naturalmente para formulações QUBO, tornando-o um candidato adequado para abordagens baseadas em QAOA.

1.2 Formulação do Problema

Dado um conjunto de processos com utilização de CPU conhecida e um número fixo de núcleos, o sistema deve atribuir cada processo a exatamente um núcleo. O objetivo de otimização é o equilíbrio de carga: a carga total atribuída a cada núcleo deve aproximar-se o mais possível da carga média global. A solução deve ainda respeitar restrições de atribuição exclusiva, garantindo que nenhum processo fica por atribuir nem é atribuído a múltiplos núcleos simultaneamente.

A mesma estrutura de atribuição exclusiva com equilíbrio de carga surge em contextos de maior escala, como a alocação de máquinas virtuais a servidores físicos em

datacenters. Nestes cenários, a complexidade combinatória cresce rapidamente com o número de entidades a alocar, tornando a exploração de abordagens baseadas em QUBO potencialmente relevante à medida que o hardware quântico evolui.

Expressar estas restrições e este objetivo numa formulação QUBO, compatível com otimizadores quânticos, constitui o problema central deste projeto.

Formulação QUBO para Escalonamento

2.1 CPU Scheduling em Sistemas Operativos

Um escalonador de CPU determina que processo ou *thread* executa em cada núcleo do processador em cada instante. Escalonadores como o *Completely Fair Scheduler* (CFS) do Linux consideram simultaneamente latência, *throughput*, prioridade, afinidade de núcleo e localização de cache, tomando decisões em microssegundos com base em heurísticas clássicas [1].

No entanto, a decisão de atribuição de processos a núcleos é fundamentalmente um problema de partição combinatorial. Dado um conjunto de N processos com cargas w_i e K núcleos, o número de atribuições possíveis cresce como K^N , tornando a procura exaustiva para instâncias de grande escala [3, 4].

Este projeto adota uma formulação simplificada deste problema: cada entidade possui uma utilização de CPU conhecida, e o objetivo é distribuí-las pelos núcleos disponíveis minimizando o desequilíbrio de carga. Esta abstração isola a componente combinatorial do escalonamento sem replicar a complexidade de um escalonador de kernel completo.

2.2 Formulação QUBO

Um problema QUBO [3, 4] define uma função de energia quadrática sobre variáveis binárias $x_i \in \{0, 1\}$:

$$E(\mathbf{x}) = \mathbf{x}^\top Q \mathbf{x} = \sum_i \sum_j Q_{ij} \cdot x_i \cdot x_j, \quad (2.1)$$

onde $Q \in \mathbb{R}^{n \times n}$ é a matriz de coeficientes e $\mathbf{x} \in \{0, 1\}^n$. O problema consiste em encontrar:

$$\mathbf{x}^* = \min_{\mathbf{x} \in \{0, 1\}^n} E(\mathbf{x}). \quad (2.2)$$

onde $Q \in \mathbb{R}^{n \times n}$ é uma matriz cujos coeficientes codificam o problema a resolver.

A energia de uma solução $\mathbf{x}$ é o valor $\mathbf{x}^\top Q \mathbf{x}$. A solução ótima $\mathbf{x}^*$ é a atribuição binária que minimiza esta energia no espaço de 2^n configurações possíveis.

Expandindo o somatório, surgem dois tipos de termos:

- **Termos diagonais** ($i = j$): como $x_i^2 = x_i$ para variáveis binárias, $Q_{ii} \cdot x_i \cdot x_i = Q_{ii} \cdot x_i$. Estes termos associam um custo individual a cada variável — um coeficiente $Q_{ii} < 0$ incentiva o otimizador quântico a escolher $x_i = 1$; um coeficiente $Q_{ii} > 0$ incentiva $x_i = 0$.
- **Termos fora da diagonal** ($i \neq j$): $Q_{ij} \cdot x_i \cdot x_j$ contribui para a energia apenas quando $x_i = x_j = 1$. Um coeficiente positivo penaliza a ativação simultânea de x_i e x_j ; um coeficiente negativo recompensa-a.

2.3 Construção da Matriz Q

Dado um conjunto de N processos com cargas w_i e K núcleos, define-se uma variável binária $x_{ij} \in \{0, 1\}$ para cada par processo-núcleo. Esta variável toma o valor 1 quando o processo i é atribuído ao núcleo j , e o valor 0 caso contrário. Na implementação, a variável x_{ij} é armazenada no índice linear $iK + j$ da matriz Q .

O objetivo original do problema é equilibrar a carga entre núcleos. Para isso, mede-se o desequilíbrio através do desvio quadrático total das cargas relativamente à carga média alvo:

$$\min \sum_{j=0}^{K-1} (L_j - L_{avg})^2, \quad (2.3)$$

onde L_j é a carga total atribuída ao núcleo j e

$$L_{avg} = \frac{W_{total}}{K} \quad (2.4)$$

é a carga média alvo entre os K núcleos.

Para obter a formulação QUBO, este termo é expandido e o resultado é codificado na matriz Q , de modo que o *solver* minimize a energia $E(\mathbf{x})$.

Assim, minimizar a energia QUBO corresponde, por construção, a procurar uma atribuição simultaneamente equilibrada e válida.

Para cada núcleo j , a carga total é dada por:

$$L_j = \phi_j + \sum_{i=0}^{N-1} w_i x_{ij}, \quad (2.5)$$

onde ϕ_j representa a carga já comprometida nesse núcleo, relevante para a decomposição iterativa descrita na Secção 5.2. Na primeira iteração, ou num QUBO sem carga fixa, tem-se $\phi_j = 0$.

Substituindo L_j no termo de um núcleo e usando $x_{ij}^2 = x_{ij}$, obtém-se:

$$\begin{aligned} (L_j - L_{avg})^2 &= \left(\phi_j + \sum_{i=0}^{N-1} w_i x_{ij} - L_{avg} \right)^2 \\ &= \underbrace{\sum_{i=0}^{N-1} d_{ij} x_{ij}}_{\text{termos diagonais}} + 2 \underbrace{\sum_{0 \leq i < k} w_i w_k x_{ij} x_{kj}}_{\text{termos de co-localização}} + \underbrace{(\phi_j - L_{avg})^2}_{\text{constante}}, \end{aligned} \quad (2.6)$$

onde

$$d_{ij} = w_i^2 + 2w_i\phi_j - 2L_{avg}w_i. \quad (2.7)$$

Somando esta expansão sobre todos os núcleos j , o desvio quadrático total iguala-se, termo a termo, à energia QUBO da Equação 2.1:

$$\sum_{j=0}^{K-1} (L_j - L_{avg})^2 = \mathbf{x}^\top Q \mathbf{x} + C, \quad (2.8)$$

onde $C = \sum_{j=0}^{K-1} (\phi_j - L_{avg})^2$ é uma constante independente de $\mathbf{x}$, e as entradas de Q correspondem diretamente aos coeficientes obtidos na expansão: d_{ij} na diagonal e $w_i w_k$ nas entradas fora da diagonal, para processos i, k atribuídos ao mesmo núcleo j . Minimizar $E(\mathbf{x})$ é, portanto, exatamente equivalente a minimizar o desvio de carga original.

Estes coeficientes são então adicionados à matriz Q . Os termos diagonais estão associados à ativação individual de uma variável x_{ij} ; os termos de co-localização aparecem quando dois processos distintos são atribuídos ao mesmo núcleo.

Além do objetivo de balanceamento, é necessário garantir que cada processo é atribuído a exatamente um núcleo. Para isso adiciona-se à matriz Q a penalidade de atribuição única:

$$P \sum_{i=0}^{N-1} \left(\sum_{j=0}^{K-1} x_{ij} - 1 \right)^2. \quad (2.9)$$

Esta penalidade segue a prática comum de converter restrições de igualdade em termos quadráticos penalizados em QUBO [4]. Neste caso, contribui com $-P$ na diagonal de cada variável e com $+P$ entre variáveis do mesmo processo em núcleos diferentes.

Combinando o objetivo de balanceamento e a penalidade de atribuição única, cada entrada da matriz é determinada pelo tipo de relação entre as duas variáveis consideradas:

$$Q[x_{ij}, x_{kl}] = \begin{cases} w_i^2 + 2w_i\phi_j - 2L_{avg}w_i - P, & \text{se } i = k \text{ e } j = l, \\ w_i w_k, & \text{se } i \neq k \text{ e } j = l, \\ P, & \text{se } i = k \text{ e } j \neq l, \\ 0, & \text{se } i \neq k \text{ e } j \neq l. \end{cases} \quad (2.10)$$

A primeira linha corresponde à contribuição individual de cada variável. A segunda linha penaliza processos distintos que partilham o mesmo núcleo, enquanto a terceira corresponde à penalidade de atribuição única entre núcleos diferentes do mesmo processo. A última linha indica que variáveis de processos e núcleos diferentes não interagem diretamente.

2.4 Exemplo Ilustrativo

Considere-se $N = 2$ processos e $K = 2$ núcleos, com cargas $w_0 = 0.3$ e $w_1 = 0.5$, sem carga fixa ($\phi_j = 0$). As variáveis ficam ordenadas como:

índice	0	1	2	3
variável	x_{00}	x_{01}	x_{10}	x_{11}

(2.11)

A carga média é $L_{avg} = (0.3 + 0.5)/2 = 0.4$.

Antes de adicionar a penalidade de atribuição única, os termos diagonais do balanceamento são:

$$\begin{aligned} Q_{0,0}^{bal} = Q_{1,1}^{bal} &= w_0^2 - 2L_{avg}w_0 = 0.09 - 2(0.4)(0.3) = -0.15, \\ Q_{2,2}^{bal} = Q_{3,3}^{bal} &= w_1^2 - 2L_{avg}w_1 = 0.25 - 2(0.4)(0.5) = -0.15. \end{aligned} \quad (2.12)$$

Para o núcleo 0, a expansão de $(L_0 - L_{avg})^2$ contém o termo de co-localização:

$$2w_0w_1x_{00}x_{10} = 0.30x_{00}x_{10}.$$

De forma análoga, a expansão de $(L_1 - L_{avg})^2$ contém:

$$2w_0w_1x_{01}x_{11} = 0.30x_{01}x_{11}.$$

Como Q é simétrica, este contributo é dividido pelas duas posições simétricas da matriz.

Assim, a matriz do objetivo de balanceamento é:

$$Q_{bal} = \begin{pmatrix} -0.15 & 0 & 0.15 & 0 \\ 0 & -0.15 & 0 & 0.15 \\ 0.15 & 0 & -0.15 & 0 \\ 0 & 0.15 & 0 & -0.15 \end{pmatrix}. \quad (2.13)$$

Com $P = 1.0$, a penalidade subtrai 1.0 a cada termo diagonal e adiciona 1.0 entre as variáveis do mesmo processo em núcleos distintos: x_{00} com x_{01} , e x_{10} com x_{11} . A matriz final fica:

$$Q = \begin{pmatrix} -1.15 & 1.0 & 0.15 & 0 \\ 1.0 & -1.15 & 0 & 0.15 \\ 0.15 & 0 & -1.15 & 1.0 \\ 0 & 0.15 & 1.0 & -1.15 \end{pmatrix}. \quad (2.14)$$

Para verificar o efeito dos termos de co-localização, comparem-se duas soluções viáveis, com $\mathbf{x}^\top = (x_{00}, x_{01}, x_{10}, x_{11})$: A, com os processos distribuídos ($\mathbf{x}_A^\top = (1, 0, 0, 1)$), e B, com ambos os processos no núcleo 0 ($\mathbf{x}_B^\top = (1, 0, 1, 0)$). Usando $E(\mathbf{x}) = \mathbf{x}^\top Q \mathbf{x}$ (2.1):

$$\begin{aligned} E_A &= Q_{0,0} + Q_{3,3} = -1.15 + (-1.15) = -\mathbf{2.30}, \\ E_B &= Q_{0,0} + Q_{2,2} + Q_{0,2} + Q_{2,0} = -1.15 + (-1.15) + 0.15 + 0.15 = -\mathbf{2.00}. \end{aligned} \quad (2.15)$$

Como $E_A < E_B$, a solução distribuída tem menor energia. A matriz penaliza, portanto, a concentração dos dois processos no mesmo núcleo, favorecendo uma distribuição mais equilibrada.

Depois de construída a matriz Q , o problema de escalonamento passa a estar expresso numa forma compatível com otimizadores QUBO. O próximo passo será resolver esta matriz, isto é, procurar a bitstring $\mathbf{x}$ que minimiza a energia $E(\mathbf{x}) = \mathbf{x}^\top Q \mathbf{x}$. Neste trabalho, essa etapa é realizada com QAOA [2].

Resolução do QUBO com QAOA

O QAOA é um algoritmo híbrido para otimização combinatória, no qual um circuito quântico é combinado com um otimizador clássico para minimizar o valor esperado de um Hamiltoniano de custo [2, 6]. Neste projeto, a implementação é feita com PennyLane [5], uma biblioteca de computação quântica em Python que suporta simulação de circuitos quânticos.

Importa realçar que a execução foi realizada em simulação, e não em hardware quântico real. Esta opção permite evitar a latência associada aos dispositivos quânticos remotos, bem como os efeitos de ruído físico. Na implementação, o *solver* tenta primeiro o backend `lightning.gpu` do PennyLane, quando há suporte para GPU. Quando este backend não está disponível, recorre a `lightning.qubit` como alternativa [7].

A abordagem implementada atribui um qubit lógico a cada par processo-núcleo. Assim, uma instância com N processos e K núcleos requer $N \cdot K$ qubits.

3.1 Conversão do QUBO em Hamiltoniano

O QAOA trabalha com um Hamiltoniano de custo H_C e não diretamente com a matriz Q . Assim, a energia QUBO

$$E(\mathbf{x}) = \mathbf{x}^\top Q \mathbf{x} \quad (3.1)$$

tem de ser convertida em forma de operadores quânticos. Os qubits, ao contrário das variáveis binárias, assumem valores $+1$ e -1 — os dois valores próprios do operador de Pauli Z_i , cujos vetores próprios são os estados da base computacional $|0\rangle$ e $|1\rangle$ respetivamente.

$$Z_i |0\rangle = +|0\rangle, \quad Z_i |1\rangle = -|1\rangle. \quad (3.2)$$

Assim, a conversão entre os dois domínios é feita pela substituição:

$$x_i = \frac{I - Z_i}{2}, \quad (3.3)$$

Desta forma, o estado $|0\rangle$, associado ao valor próprio $+1$ de Z_i , corresponde a $x_i = 0$, enquanto o estado $|1\rangle$, associado ao valor próprio -1 , corresponde a $x_i = 1$ [8].

Assim, um termo linear da diagonal do QUBO transforma-se em:

$$Q_{ii} x_i = Q_{ii} \frac{I - Z_i}{2} = \frac{Q_{ii}}{2} I - \frac{Q_{ii}}{2} Z_i. \quad (3.4)$$

Para os termos quadráticos, considerando $q_{ij} = Q_{ij} + Q_{ji}$, obtém-se:

$$q_{ij}x_i x_j = q_{ij} \frac{I - Z_i}{2} \frac{I - Z_j}{2} = \frac{q_{ij}}{4} (I - Z_i - Z_j + Z_i Z_j). \quad (3.5)$$

Desta forma, o Hamiltoniano de custo — que passa a definir a energia associada a cada estado — pode ser escrito como:

$$H_C = cI + \sum_i h_i Z_i + \sum_{i < j} J_{ij} Z_i Z_j, \quad (3.6)$$

onde c é uma constante, h_i são os coeficientes lineares associados a cada qubit, e J_{ij} representa a interação entre pares de qubits.

Como este Hamiltoniano é composto apenas por termos diagonais na base computacional, cada bitstring $|x\rangle$ é um vetor próprio de H_C , com um valor próprio $E(x)$. Assim:

$$H_C |x\rangle = E(x) |x\rangle. \quad (3.7)$$

Na computação quântica, as operações aplicadas ao estado do sistema são descritas por operadores unitários. No QAOA, o Hamiltoniano de custo é usado através da operação unitária de custo [2]:

$$U_C(\gamma) = e^{-i\gamma H_C}, \quad (3.8)$$

onde γ é um parâmetro variacional. Uma vez que $|x\rangle$ é vetor próprio de H_C , ao expandir $U_C(\gamma)$ em série de Taylor, a aplicação desta unitária resulta em:

$$U_C(\gamma) |x\rangle = e^{-i\gamma E(x)} |x\rangle. \quad (3.9)$$

Em computação quântica, cada bitstring tem uma amplitude associada, que determina a probabilidade dessa bitstring ser medida. Para além do seu módulo, essa amplitude também possui uma fase. O fator $e^{-i\gamma E(x)}$ altera essa fase de acordo com a energia da bitstring, sem alterar diretamente a sua probabilidade de medição.

Assim, a camada de custo não altera as probabilidades de medição. Em vez disso, introduz fases diferentes nas bitstrings, de acordo com a sua energia. Estas fases tornam úteis quando é aplicada a camada de mistura, que altera a distribuição das amplitudes e, conseqüentemente, as probabilidades de medir cada solução.

A camada de mistura é definida por outro Hamiltoniano H_M e pelo operador unitário

$$U_M(\beta) = e^{-i\beta H_M}, \quad (3.10)$$

onde β é também um parâmetro variacional. Enquanto U_C traduz informação sobre a qualidade das soluções através das fases, U_M permite explorar novas soluções ao misturar amplitudes entre estados da base computacional.

3.2 Mixer X e XY

O Hamiltoniano H_M presente em (3.10) é designado por *mixer*. No QAOA, o *mixer* é responsável por distribuir amplitude entre estados da base computacional [2, 9]. Para este trabalho foram considerados dois tipos: X e XY.

A diferença entre eles está no espaço de soluções explorado. O *mixer X* atua sobre todos os qubits individualmente, permitindo explorar todo o espaço binário. Já o *mixer XY* é adaptado à restrição de atribuição única, pois atua apenas entre qubits que representam núcleos alternativos para o mesmo processo [10, 11].

3.2.1 Mixer X

O *mixer X* é o padrão da formulação do QAOA [2]. O seu Hamiltoniano de forma simplificada é:

$$H_M^X = \sum_i X_i \quad (3.11)$$

onde X_i é o operador de Pauli X aplicado ao qubit i .

Este mixer é aplicado numa superposição de estados uniforme, aplicando *Hadamard gates* no estado inicial.

$$|s\rangle = \frac{1}{\sqrt{2^n}} \sum_{x \in \{0,1\}^n} |x\rangle, \quad (3.12)$$

onde $n = N \cdot K$ é o número total de qubits. Assim, o circuito começa numa superposição sobre todos os bitstrings possíveis.

No entanto, para o problema do escalonamento este espaço contém estados inválidos. Por exemplo, para um único processo e dois núcleos, existem quatro estados possíveis:

$$|00\rangle, |01\rangle, |10\rangle, |11\rangle. \quad (3.13)$$

Assumindo que cada posição representa a atribuição do processo a um núcleo, apenas dois destes estados são válidos, devido à restrição de atribuição única:

$$|10\rangle, |01\rangle. \quad (3.14)$$

Por isso, com o mixer X, a validade das soluções depende da penalidade de atribuição única incluída no QUBO.

3.2.2 Mixer XY

O *mixer XY* é mais adequado à estrutura deste problema, porque atua apenas entre os qubits que representam os possíveis núcleos de um mesmo processo. Para cada processo i , existe um bloco de K qubits:

$$x_{i0}, x_{i1}, \dots, x_{i,K-1}. \quad (3.15)$$

O mixer XY aplica operações entre pares de qubits dentro desse bloco, que têm a forma:

$$X_{ij}X_{ik} + Y_{ij}Y_{ik} \quad (3.16)$$

onde j e k representam dois núcleos distintos associados ao mesmo processo i . Esta operação permite transferir amplitude entre atribuições do mesmo processo, por exemplo:

$$|10\rangle \leftrightarrow |01\rangle. \quad (3.17)$$

Esta operação preserva o número de bits ativos no bloco. Assim, partindo de um estado válido, como $|10\rangle$ ou $|01\rangle$, o mixer XY não cria estados inválidos como $|00\rangle$ ou $|11\rangle$ [10, 11].

Deste modo, embora o circuito continue a ter $N \cdot K$ qubits e o espaço binário total tenha dimensão

$$2^{N \cdot K}, \quad (3.18)$$

o mixer XY restringe o subespaço que contém

$$K^N \quad (3.19)$$

atribuições válidas. Por esta razão, este *mixer* é particularmente adequado para o problema de atribuição de processos a núcleos.

3.3 Circuito Variacional e Otimização Clássica

Depois de definidas as camadas de custo e de mistura, o circuito QAOA é construído alternando estas duas operações durante p camadas [2]:

$$|\psi(\boldsymbol{\gamma}, \boldsymbol{\beta})\rangle = U_M(\boldsymbol{\beta}_p)U_C(\boldsymbol{\gamma}_p) \cdots U_M(\boldsymbol{\beta}_1)U_C(\boldsymbol{\gamma}_1)|\psi_0\rangle. \quad (3.20)$$

Os vetores $\boldsymbol{\gamma} = (\gamma_1, \dots, \gamma_p)$ e $\boldsymbol{\beta} = (\beta_1, \dots, \beta_p)$ são parâmetros variacionais. O seu valor não é fixado manualmente; é ajustado por um otimizador clássico de forma a minimizar o valor esperado do Hamiltoniano de custo:

$$\min_{\boldsymbol{\gamma}, \boldsymbol{\beta}} \langle \psi(\boldsymbol{\gamma}, \boldsymbol{\beta}) | H_C | \psi(\boldsymbol{\gamma}, \boldsymbol{\beta}) \rangle. \quad (3.21)$$

Este valor esperado corresponde à energia média das bitstrings que podem ser obtidas ao medir o estado final do circuito. Como o Hamiltoniano de custo codifica a função objetivo do problema, minimizar $\langle H_C \rangle$ significa ajustar o circuito para aumentar a probabilidade de obter soluções com menor custo.

O processo é híbrido: para um dado conjunto de parâmetros, o circuito quântico é executado e o valor esperado do Hamiltoniano de custo é estimado a partir dos resultados de medição. De seguida, um otimizador clássico usa essa informação para escolher novos valores de γ e β . Este ciclo repete-se até serem encontrados parâmetros que produzam uma distribuição de resultados mais favorável.

A otimização é feita com o otimizador Adam do PennyLane [12], durante um número configurável de iterações. Em cada iteração, o circuito é avaliado, o valor esperado de H_C é calculado, e os parâmetros são atualizados para reduzir essa energia esperada.

3.4 Medição, Descodificação e Seleção da Solução

Após a otimização dos parâmetros variacionais, o circuito QAOA final define uma distribuição de probabilidades sobre os bitstrings possíveis. Os estados são ordenados por probabilidade decrescente, sendo retidos como candidatos apenas os `top_k` estados mais prováveis.

Cada candidato é então decodificado e validado segundo a restrição de atribuição única: para cada processo, exatamente uma das K variáveis associadas aos núcleos deve estar ativa. Se nenhuma variável estiver ativa, ou se estiver ativa mais do que uma, o bitstring é considerado inválido.

Para cada bitstring viável, a energia clássica é recalculada diretamente a partir da matriz QUBO:

$$E(\mathbf{x}) = \mathbf{x}^\top \mathbf{Q} \mathbf{x}. \quad (3.22)$$

Quando o conjunto `top_k` contém pelo menos um candidato viável, o solver seleciona aquele que apresenta a menor energia QUBO. Desta forma, a etapa quântica gera uma distribuição candidata, enquanto a etapa clássica final garante que a solução atribuída é válida e energeticamente favorável.

Decomposição Adaptativa

Como descrito anteriormente, cada par entidade-núcleo é representado por uma variável binária. Para N entidades e K núcleos, o número de qubits lógicos necessários é:

$$n_q = N \cdot K. \quad (4.1)$$

Como n_q cresce linearmente com N , instâncias com muitas entidades rapidamente excedem os limites práticos de escala do hardware quântico atual, marcado por baixa contagem de qubits e elevadas taxas de erro [13]. Por exemplo, para $K = 8$ núcleos e $N = 50$ entidades, são necessários $n_q = 400$ qubits, um valor que já excede a capacidade da maioria dos dispositivos atuais.

Para contornar esta limitação, a plataforma inclui uma etapa de decomposição adaptativa: em vez de considerar todas as entidades em simultâneo, estas são agrupadas em conjuntos de menor dimensão, designados *bundles*, formados com base na semelhança entre processos, considerando a sua carga de CPU e memória usada.

Esta abordagem vem que o QAOA opera sobre entidades e as suas cargas agregadas, sem distinguir se cada entidade corresponde a um processo isolado ou a um grupo de procesos, basta que os processos dentro do mesmo bundle sejam suficientemente homogêneos em carga que o bundle comporta-se como uma entidade equivalente e a qualidade da solução não é degradada.

A afinidade de memória complementa a decomposição para evitar juntar processos com perfis de memória muito diferentes.

Considere-se um *snapshot* com um conjunto de processos $\mathcal{P} = \{p_1, p_2, \dots, p_N\}$ e K núcleos disponíveis. Em primeiro lugar, os processos de tempo real são separados dos restantes, definindo-se $\mathcal{P}_N = \mathcal{P} \setminus \mathcal{P}_{RT}$.

Os processos em $\mathcal{P}_{RT}$ mantêm a sua atribuição fixa ao núcleo atual, uma vez que processos de tempo real têm requisitos de latência estritos que desaconselham a sua reatribuição. A decomposição adaptativa é aplicada apenas aos processos normais $\mathcal{P}_N$.

Dado um limite máximo de qubits q_{max} , define-se o número máximo de processos representáveis por *bundle* como:

$$m = \left\lfloor \frac{q_{max}}{K} \right\rfloor. \quad (4.2)$$

Para $N_N = |\mathcal{P}_N|$ processos normais, o número inicial de *bundles* é calculado por:

$$B = \min \left(\left\lceil \frac{N_N}{m} \right\rceil, N_N \right). \quad (4.3)$$

Quando $B \geq N_N$, não é necessário agrupar processos, pelo que cada processo é representado por um *bundle* individual.

Caso contrário, os processos são agrupados recorrendo a *spectral clustering* [14]. Ao contrário de métodos como o K-Means, que tendem a formar grupos de tamanho semelhante e forma regular, o *spectral clustering* consegue detetar agrupamentos naturais nos dados mesmo quando estes têm formas irregulares — uma propriedade relevante em *snapshots* reais, onde os processos podem formar grupos de diferentes tamanhos e densidades. Para isso, é necessário primeiro construir uma matriz de afinidade entre processos.

Para cada processo p_i , começa-se por calcular uma carga efetiva de CPU, reduzindo o peso de processos que passam parte significativa do tempo em espera por I/O:

$$\tilde{w}_i = w_i (1 - \alpha_{io} r_i), \quad (4.4)$$

onde w_i representa a carga de CPU observada, $r_i \in [0, 1]$ é a fração de tempo em espera por I/O, e α_{io} controla o impacto dessa espera na carga efetiva.

Esta correção é relevante porque melhora a qualidade da informação usada para construir a matriz de afinidade entre processos: ao distinguir processos *CPU-bound* de processos *I/O-bound*, permite que a decomposição agrupe processos com perfis semelhantes de forma mais precisa, melhorando a qualidade da decomposição.

Cada processo é então representado por um vetor de características com duas componentes:

$$\mathbf{f}_i = \begin{bmatrix} \tilde{w}_i \\ \text{rss}_i \end{bmatrix}, \quad (4.5)$$

onde rss_i corresponde à memória residente do processo.

Com base nas características de cada processo, é construída uma matriz de afinidade A , que mede a semelhança entre pares de processos. O objetivo é agrupar processos com perfis semelhantes — por exemplo, processos com pouca utilização de CPU e elevada utilização de memória devem ficar no mesmo *bundle*. Para isso, são consideradas duas componentes: utilização de CPU e memória (a utilização de I/O também é aplicada indiretamente através do peso efetivo $\tilde{w}_i$). A distância entre dois processos em cada componente é medida pelo quadrado da diferença dos valores normalizados:

$$d_{ij}^{cpu} = (\hat{f}_{i,cpu} - \hat{f}_{j,cpu})^2, \quad d_{ij}^{mem} = (\hat{f}_{i,mem} - \hat{f}_{j,mem})^2. \quad (4.6)$$

Distâncias menores indicam processos mais semelhantes. No entanto, o *spectral clustering* opera sobre afinidades, não sobre distâncias: valores altos devem indicar processos semelhantes, e não o contrário. Para converter estas distâncias em afinidades, aplica-se um kernel RBF, fazendo com que a afinidade decresça rapidamente à medida que os processos se tornam mais distintos:

$$A_{ij}^{cpu} = \exp\left(-\frac{d_{ij}^{cpu}}{2\sigma_{cpu}^2}\right), \quad A_{ij}^{mem} = \exp\left(-\frac{d_{ij}^{mem}}{2\sigma_{mem}^2}\right). \quad (4.7)$$

Os parâmetros σ_{cpu} e σ_{mem} são calculados como a mediana das distâncias entre todos os pares de processos.

A matriz de afinidade final combina as duas componentes:

$$A_{ij} = \alpha A_{ij}^{cpu} + (1 - \alpha) A_{ij}^{mem}, \quad i \neq j, \quad (4.8)$$

O parâmetro α controla a importância relativa da utilização de CPU face à utilização de memória.

O algoritmo devolve uma função de atribuição:

$$g : \mathcal{P}_N \rightarrow \{0, 1, \dots, B - 1\}, \quad (4.9)$$

onde $g(p_i)$ indica o *bundle* atribuído ao processo p_i . Para cada *bundle* b , são calculadas as grandezas agregadas de CPU e memória:

$$W_b = \sum_{p_i: g(p_i)=b} \tilde{w}_i, \quad R_b = \sum_{p_i: g(p_i)=b} r_{ss_i}. \quad (4.10)$$

Cada *bundle* passa então a ser tratado como uma única entidade na formulação QUBO, com peso W_b e memória agregada R_b .

4.0.1 Verificações de Qualidade e Divisão Recursiva

O clustering inicial pode produzir *bundles* problemáticos: um bundle pode ser internamente heterogéneo, ou concentrar carga excessiva de CPU ou memória. Em qualquer destes casos, representá-lo como uma única entidade no QUBO introduziria imprecisão na formulação. Por isso, após o clustering inicial, cada *bundle* é sujeito a três verificações:

- **Heterogeneidade interna:** medida pelo coeficiente de variação da carga dos processos no bundle,

$$CV_b = \frac{\text{std}(\{\tilde{w}_i : g(p_i) = b\})}{\text{mean}(\{\tilde{w}_i : g(p_i) = b\})}. \quad (4.11)$$

Um bundle é considerado heterogéneo quando CV_b excede um limiar configurável.

- **Peso outlier:** a carga agregada W_b é comparada com a distribuição dos restantes *bundles* através do z-score,

$$z_b = \frac{W_b - \mu_W}{\sigma_W}, \quad (4.12)$$

onde μ_W e σ_W são a média e o desvio padrão das cargas agregadas. Um bundle é considerado demasiado pesado quando $|z_b|$ excede um limiar configurável.

- **Pressão de memória:** a memória agregada R_b é comparada com um limiar configurável.

Se um *bundle* falhar pelo menos uma destas verificações e contiver mais do que um processo, é dividido em dois sub-bundles usando K-Means sobre os pesos efetivos de CPU. Cada sub-bundle é depois avaliado recursivamente com as mesmas verificações. A recursão termina quando uma das seguintes condições for satisfeita: o bundle contém apenas um processo; foi atingida a profundidade máxima de recursão; ou os pesos dos processos são praticamente iguais, caso em que os processos são separados em *bundles* únicos.

4.0.2 Exemplo da Formação de Bundles

Considere-se um sistema com $K = 2$ núcleos, um limite $q_{\max} = 4$ e oito processos normais. Para simplificar o exemplo, assume-se que nenhum processo está em espera por I/O, pelo que $\tilde{w}_i = w_i$.

Processo	Carga de CPU	RSS (MB)
p_1	0.10	100
p_2	0.11	110
p_3	0.20	200
p_4	0.21	210
p_5	0.30	300
p_6	0.31	310
p_7	0.40	400
p_8	0.41	410

Tabela 4.1: Processos usados no exemplo de formação de bundles.

O número de processos de referência por *bundle* é

$$m = \left\lfloor \frac{q_{\max}}{K} \right\rfloor = \left\lfloor \frac{4}{2} \right\rfloor = 2. \quad (4.13)$$

Consequentemente, o número inicial de *bundles* é

$$B = \left\lceil \frac{8}{2} \right\rceil = 4. \quad (4.14)$$

Observando a tabela, verifica-se perfis de CPU e memória muito semelhantes, como por exemplo, p_1 e p_2 que diferem apenas em 0.01 de carga de CPU e 10 MB de memória. A sua afinidade é, portanto, superior à existente entre processos pertencentes a pares diferentes.

Após a normalização, obtêm-se os valores:

$$\begin{aligned} \hat{\mathbf{f}}_1 &= (-1.385, -1.385), & \hat{\mathbf{f}}_2 &= (-1.296, -1.296), \\ \hat{\mathbf{f}}_3 &= (-0.491, -0.491), & \hat{\mathbf{f}}_4 &= (-0.402, -0.402), \\ \hat{\mathbf{f}}_5 &= (0.402, 0.402), & \hat{\mathbf{f}}_6 &= (0.491, 0.491), \\ \hat{\mathbf{f}}_7 &= (1.296, 1.296), & \hat{\mathbf{f}}_8 &= (1.385, 1.385). \end{aligned} \quad (4.15)$$

A proximidade entre processos é depois convertida numa afinidade através do kernel RBF. Neste exemplo, algumas das afinidades obtidas são:

$$\begin{aligned} A_{12} \approx 0.992, \quad A_{34} \approx 0.992, \\ A_{56} \approx 0.992, \quad A_{78} \approx 0.992. \end{aligned} \quad (4.16)$$

Estes valores mostram uma afinidade muito elevada entre os processos de cada par. Em comparação, as afinidades entre pares diferentes são inferiores, por exemplo,

$$A_{23} \approx A_{45} \approx A_{67} \approx 0.512, \quad A_{18} < 0.001. \quad (4.17)$$

O *spectral clustering* interpreta a matriz de afinidade como um grafo: cada processo corresponde a um vértice e cada afinidade representa o peso da ligação entre dois processos:

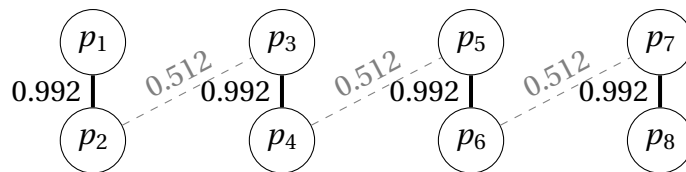


Figura 4.1: Representação simplificada do grafo de afinidade. Algumas ligações foram omitidas para facilitar a leitura.

O *spectral clustering* analisa o grafo de afinidade para identificar processos fortemente relacionados. Os processos são depois separados em $B = 4$ clusters através do K-Means.

Para o exemplo considerado, são obtidos os seguintes *bundles*:

$$\begin{aligned} B_0 = \{p_1, p_2\}, \quad B_1 = \{p_3, p_4\}, \\ B_2 = \{p_5, p_6\}, \quad B_3 = \{p_7, p_8\}. \end{aligned} \quad (4.18)$$

As cargas agregadas resultantes são:

Bundle	Processos	W_b	R_b (MB)
B_0	p_1, p_2	$0.10 + 0.11 = 0.21$	210
B_1	p_3, p_4	$0.20 + 0.21 = 0.41$	410
B_2	p_5, p_6	$0.30 + 0.31 = 0.61$	610
B_3	p_7, p_8	$0.40 + 0.41 = 0.81$	810

Tabela 4.2: Bundles e respetivas grandezas agregadas.

Nenhum destes *bundles* requer divisão adicional, uma vez que os seus processos apresentam cargas semelhantes e os valores agregados não excedem os limites de qualidade considerados.

A partir deste ponto, cada *bundle* passa a ser tratado como uma entidade individual. Assim, o QUBO global possui

$$NK = 4 \times 2 = 8 \quad (4.19)$$

variáveis binárias.

Decomposição em Sub-QUBOs

Após a decomposição adaptativa, o problema deixa de ser expresso em termos de processos individuais e passa a ser definido sobre um conjunto de entidades. Cada entidade pode corresponder a um processo isolado ou a um *bundle*. Seja

$$\mathcal{E} = \{e_0, e_1, \dots, e_{N-1}\} \quad (5.1)$$

o conjunto de entidades resultante, onde cada entidade e_i tem peso de CPU w_i e memória residente r_i .

Para K núcleos, a formulação continua a exigir $n_q = N \cdot K$ qubits lógicos. Assim, mesmo depois do agrupamento em *bundles*, o problema pode continuar a exceder o limite configurado q_{max} . Quando $N \cdot K \leq q_{max}$, a plataforma resolve um único QUBO. Quando $N \cdot K > q_{max}$, é usada uma pipeline iterativa baseada em *sub-QUBOs*.

5.1 Construção do QUBO Global

O primeiro passo da pipeline iterativa consiste em construir uma matriz QUBO global Q^G sobre todas as entidades de $\mathcal{E}$. Esta matriz é construída com a mesma formulação apresentada anteriormente.

Para cada entidade e_i e núcleo k , a variável binária correspondente é:

$$x_{ik} = \begin{cases} 1, & \text{se a entidade } e_i \text{ for atribuída ao núcleo } k, \\ 0, & \text{caso contrário.} \end{cases} \quad (5.2)$$

No entanto, quando o número de variáveis excede q_{max} , esta matriz não é enviada diretamente para o *solver*. Em vez disso, é usada como referência para construir sucessivos *sub-QUBOs*.

5.2 Extração de um Sub-QUBO

As entidades construídas na decomposição adaptativa são divididas em grupos

$$\mathcal{G}_0, \mathcal{G}_1, \dots, \mathcal{G}_{T-1}, \quad (5.3)$$

onde cada grupo contém no máximo

$$m = \left\lfloor \frac{q_{max}}{K} \right\rfloor \quad (5.4)$$

entidades.

Estes grupos são feitos através de duas estratégias:

- **WEIGHT_DESCENDING**: ordena os *bundles* por carga de CPU decrescente;
- **COUPLING_DESCENDING**: começa pelo *bundle* com maior magnitude de carga e adiciona os *bundles* com maior acoplamento, dado por $2W_i W_j$, aos que já pertencem ao grupo. Desta forma, as interações mais fortes permanecem dentro do mesmo *sub-QUBO*.

Para cada grupo $\mathcal{G}_t$, é extraída de Q^G a submatriz associada às variáveis das entidades desse grupo.

Este processo preserva os termos internos entre entidades do mesmo grupo. No entanto, os termos com entidades já resolvidas em iterações anteriores deixam de aparecer, porque essas entidades já têm uma atribuição fixa. Para incorporar o seu efeito, a implementação mantém um vetor de carga acumulada por núcleo:

$$\boldsymbol{\phi}^{(t)} = (\phi_0^{(t)}, \phi_1^{(t)}, \dots, \phi_{K-1}^{(t)}), \quad (5.5)$$

inicializado com a carga dos processos de tempo real $\mathcal{P}_{RT}$. Para uma entidade e_i do grupo atual e um núcleo k , a diagonal do *sub-QUBO* associado à atribuição (e_i, k) é corrigida por:

$$Q_{(e_i, k), (e_i, k)}^{(t)} \leftarrow Q_{(e_i, k), (e_i, k)}^{(t)} + 2w_i \phi_k^{(t)}. \quad (5.6)$$

Este termo vem da expansão do balanceamento de carga quando já existe carga fixa no núcleo k :

$$\left(\phi_k^{(t)} + \sum_{e_i \in \mathcal{G}_t} w_i x_{ik} - L_{avg} \right)^2. \quad (5.7)$$

Desta forma, as decisões tomadas nos *sub-QUBOs* anteriores influenciam a energia do *sub-QUBO* atual através de termos lineares.

5.3 Resolução Iterativa

Cada *sub-QUBO* é resolvido sequencialmente com o mesmo *solver* QAOA usado no caso direto. Seja $a_i^{(t)}$ o núcleo atribuído à entidade $e_i \in \mathcal{G}_t$ na iteração t . Depois de resolver um *sub-QUBO* viável, as atribuições são fixadas e a carga acumulada é atualizada:

$$\phi_k^{(t+1)} = \phi_k^{(t)} + \sum_{\substack{e_i \in \mathcal{G}_t \\ a_i^{(t)} = k}} w_i. \quad (5.8)$$

O processo repete-se até todos os grupos terem sido resolvidos. Se algum *sub-QUBO* não produzir uma atribuição viável, a pipeline termina antes de atualizar as atribuições finais e o vetor de carga, evitando propagar uma solução inválida para as iterações seguintes.

No final, as atribuições parciais são combinadas numa única solução global. Esta solução é validada reconstruindo a bitstring correspondente e avaliando a energia na formulação QUBO global. A decomposição em *sub-QUBOs* não é equivalente a resolver o QUBO completo de uma só vez; trata-se de uma aproximação iterativa que reduz a dimensão de cada problema resolvido, propagando as decisões anteriores através do vetor ϕ .

5.3.1 Exemplo de Extração dos Sub-QUBOs

Retomando os quatro *bundles* do exemplo anterior, as suas cargas agregadas são:

$$W_0 = 0.21, \quad W_1 = 0.41, \quad W_2 = 0.61, \quad W_3 = 0.81. \quad (5.9)$$

Como existem quatro *bundles* e dois núcleos, o QUBO global possui $4 \times 2 = 8$ variáveis. Este valor excede o limite $q_{\max} = 4$.

Cada *sub-QUBO* pode incluir, no máximo,

$$\left\lfloor \frac{q_{\max}}{K} \right\rfloor = \left\lfloor \frac{4}{2} \right\rfloor = 2 \quad (5.10)$$

bundles. Aplicando a ordenação WEIGHT_DESCENDING, são formados os seguintes grupos:

$$\mathcal{G}_0 = \{B_3, B_2\}, \quad \mathcal{G}_1 = \{B_1, B_0\}. \quad (5.11)$$

Esta divisão não cria novos *bundles*. Cada grupo determina apenas as variáveis de Q^G que serão incluídas no respetivo *sub-QUBO*.

5.3.1.1 Primeiro sub-QUBO

O primeiro grupo contém os *bundles* B_3 e B_2 . As suas variáveis de atribuição são

$$\mathcal{V}_0 = \{x_{3,0}, x_{3,1}, x_{2,0}, x_{2,1}\}. \quad (5.12)$$

O primeiro *sub-QUBO* corresponde à submatriz de Q^G associada a estas variáveis:

$$Q^{(0)} = Q^G[\mathcal{V}_0, \mathcal{V}_0]. \quad (5.13)$$

Como o exemplo não inclui processos de tempo real, a carga acumulada inicial é

$$\phi^{(0)} = (0, 0). \quad (5.14)$$

Suponha-se que a solução obtida atribui B_3 ao núcleo 0 e B_2 ao núcleo 1. A carga acumulada passa então a ser

$$\phi^{(1)} = (W_3, W_2) = (0.81, 0.61). \quad (5.15)$$

5.3.1.2 Segundo sub-QUBO

O segundo grupo contém os *bundles* B_1 e B_0 , sendo consideradas as variáveis

$$\mathcal{V}_1 = \{x_{1,0}, x_{1,1}, x_{0,0}, x_{0,1}\}. \quad (5.16)$$

A submatriz correspondente é extraída de Q^G . Antes da sua resolução, a carga acumulada no primeiro *sub-QUBO* é incorporada nos elementos diagonais. Os acréscimos aplicados são:

Variável	Acréscimo diagonal
$x_{1,0}$	$2(0.41)(0.81) = 0.6642$
$x_{1,1}$	$2(0.41)(0.61) = 0.5002$
$x_{0,0}$	$2(0.21)(0.81) = 0.3402$
$x_{0,1}$	$2(0.21)(0.61) = 0.2562$

Tabela 5.1: Bias introduzido no segundo sub-QUBO pela carga acumulada.

Como o núcleo 0 possui inicialmente uma carga superior, as variáveis que representam novas atribuições a esse núcleo recebem um acréscimo maior.

Neste caso, a solução atribui B_1 ao núcleo 1 e B_0 ao núcleo 0. A carga final é, portanto,

$$\begin{aligned} \phi_0^{(2)} &= W_3 + W_0 = 0.81 + 0.21 = 1.02, \\ \phi_1^{(2)} &= W_2 + W_1 = 0.61 + 0.41 = 1.02. \end{aligned} \quad (5.17)$$

A solução final obtida é:

Núcleo	Bundles	Processos	Carga total
0	B_3, B_0	p_7, p_8, p_1, p_2	1.02
1	B_2, B_1	p_5, p_6, p_3, p_4	1.02

Tabela 5.2: Resultado final do exemplo de resolução por sub-QUBOs.

Arquitetura e Implementação

O ponto de entrada do sistema é o `SchedulingEngine`, definido em `src/main.py`. Este componente centraliza a inicialização das configurações, instancia os módulos necessários e determina o fluxo de execução.

Esta camada não implementa diretamente a lógica de formulação, decomposição ou resolução. O seu papel é orquestrar os componentes da arquitetura, mantendo cada responsabilidade encapsulada no respetivo módulo.

A partir daqui, a arquitetura foi desenhada com dois objetivos principais: modularidade e separação de responsabilidades. A modularidade permite trocar a origem dos dados, comparar estratégias de resolução e introduzir novas formas de decomposição sem alterar toda a pipeline. Já as responsabilidades foram distribuídas por quatro componentes: captura de processos, formulação QUBO, resolução por *solvers* e validação/análise.

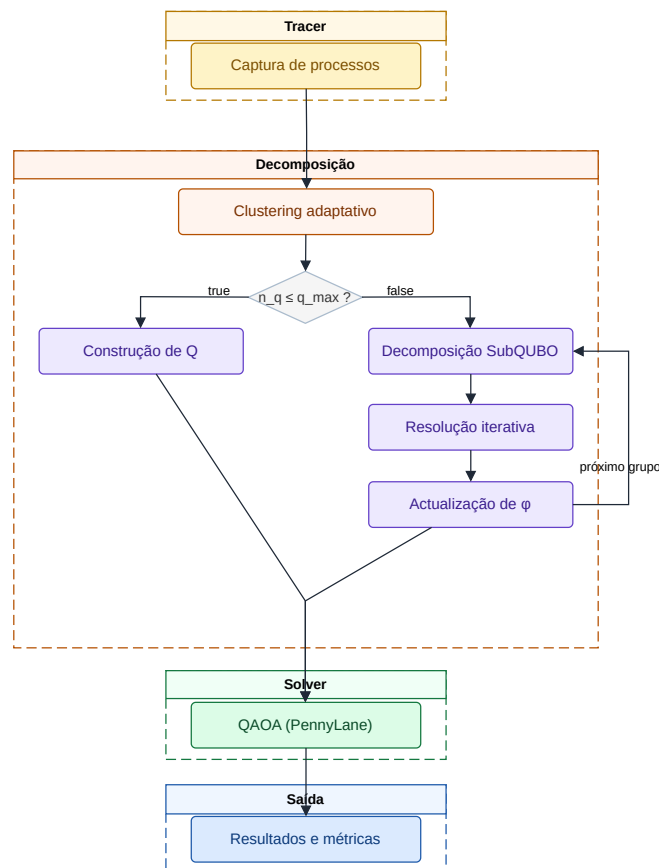


Figura 6.1: Visão geral do fluxo da arquitetura.

A Figura 6.1 ilustra o fluxo completo da arquitetura, desde a aquisição dos processos até à produção dos resultados.

O fluxo de execução começa pelo Tracer. O objectivo desta camada é produzir uma descrição dos processos a escalonar (cargas de CPU, memória utilizada e núcleo atual), que serve de base a todo o resto da pipeline. Antes de poder ser entregue ao *solver*, essa descrição tem de ser traduzida para uma representação matemática que o algoritmo de optimização compreenda: uma instância QUBO. A zona de decomposição é responsável por essa tradução e, consoante a dimensão do problema, segue um de dois caminhos:

- quando $n_q \leq q_{max}$, é construída uma única matriz Q e a instância é resolvida directamente pelo *solver*;
- quando $n_q > q_{max}$, os processos são primeiro agrupados em *bundles* e o problema é decomposto em *sub-QUBOs*

Independentemente do ramo seguido, o resultado é uma ou mais instâncias QUBO entregues ao *solver*. A solução obtida é depois validada, decodificada em atribuições processo-núcleo e convertida em métricas e ficheiros de saída.

6.1 Da Visão Geral à Implementação

Agora que temos uma visão geral do sistema, podemos ler a implementação através das interfaces de dados. Em vez de cada módulo conhecer todos os detalhes dos módulos anteriores, o sistema faz os dados atravessarem representações intermédias de dados. Esta escolha reduz o acoplamento: o *solver* não precisa de saber se os pesos vieram de uma captura real, de um cenário experimental ou de um *bundle*; recebe apenas uma `QUBOInstance`.

O fluxo de implementação pode ser resumido como uma sequência de conversão de dados:

$$\begin{aligned} \text{SystemSnapshot} &\rightarrow \text{Workload} \rightarrow \text{QUBOInstance} \\ &\rightarrow \text{SolverResult} \rightarrow \text{SchedulingOutput} \end{aligned}$$

Neste fluxo, a primeira convergência ocorre na entrada. Os dados podem vir de:

- *presets*
- uma captura real feita por `ProcessTracer.trace()`
- cenários experimentais definidos em ficheiros TOML

Apesar de estas origens serem diferentes, todas são convertidas para uma representação comum, `SystemSnapshot`. Esta estrutura permite definir vários pontos de entrada, sem ser necessário mudar a implementação do sistema.

A etapa seguinte é a camada de decomposição que transforma o snapshot num `Workload`. Esta é uma abstração importante: a partir daqui, o sistema deixa de trabalhar directamente com processos do sistema operativo e passa a trabalhar com entidades escalonáveis, sejam elas processos ou *bundles* de processos.

Quando há decomposição adaptativa, `AdaptiveCluster.decompose()` produz um `ClusteredSnapshot` como passo intermédio; quando não há agrupamento, o próprio `SystemSnapshot` é convertido diretamente. Nos dois casos, o resultado entregue à formulação QUBO tem a mesma forma de *Workload*.

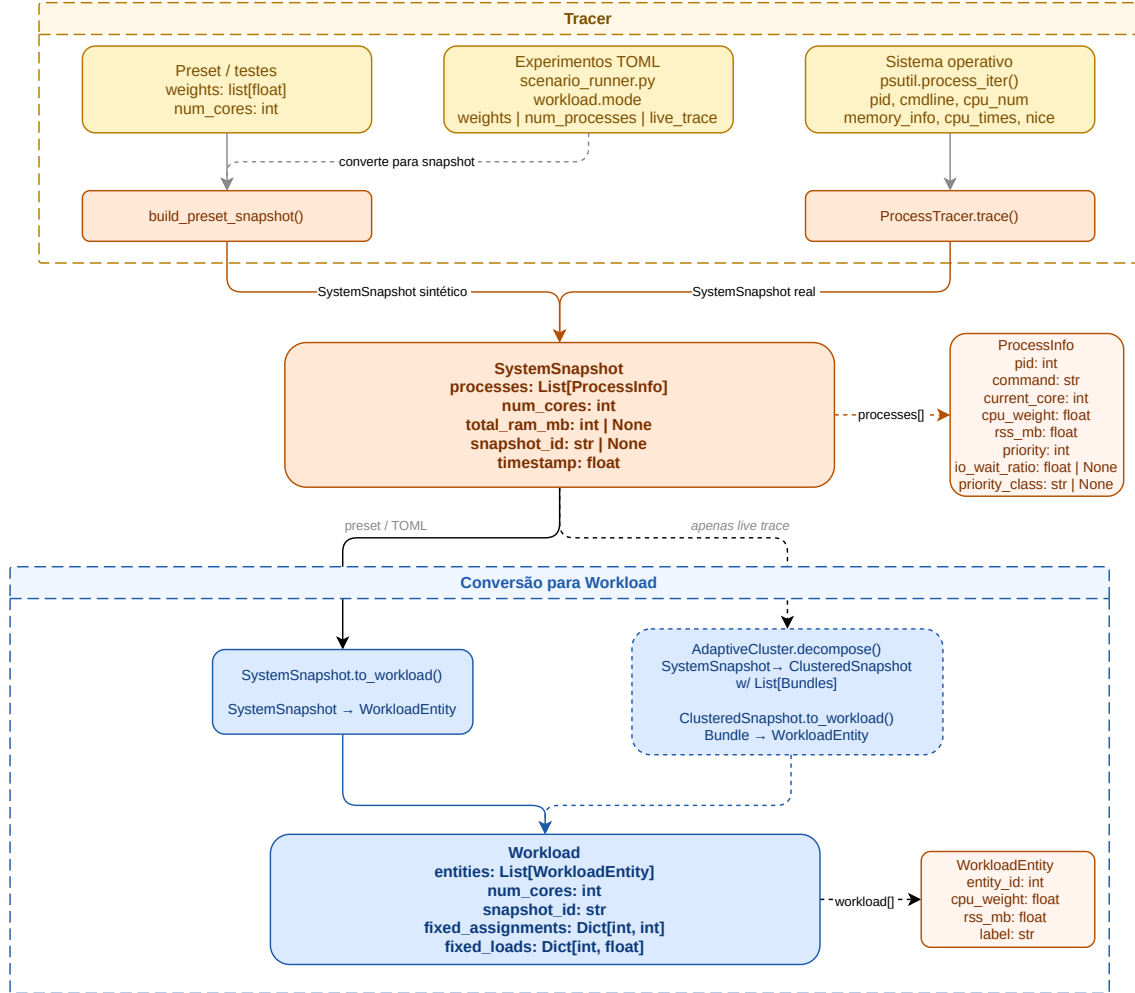


Figura 6.2: Interfaces de dados entre aquisição, `SystemSnapshot` e `Workload`.

É neste ponto que a decisão $N \times K \leq q_{max}$ da visão geral aparece no código, originando dois modos de execução:

- se o limite de qubits permitir, a `DefaultPipeline` constrói uma `QUBOInstance` única através do `CoreAssignmentBuilder` e resolve-a diretamente;
- se o limite de qubits for excedido, é executada a `IterativePipeline`. Neste modo, é construída uma `QUBOInstance` global e o `SubQUBOComposer` particiona o problema, extraindo em cada iteração uma nova `QUBOInstance` correspondente a um sub-QUBO.

Assim, as duas pipelines preservam a estrutura de dados que o *solver* espera receber. A partir daqui, as `QUBOInstance` são entregues a um dos *solvers*:

- o *solver* `PennyLane` implementa o QAOA;

- o *solver* de *annealing* serve como referência heurística;
- o brute-force fornece uma referência exata para instâncias pequenas.

Como todos usam a mesma entrada e devolvem o mesmo tipo de resultado, *SolverResult*, podem ser comparados facilmente.

Por fim, o *SolverValidator* recebe um *SolverResult* e avalia a solução candidata. A saída final distingue os dois caminhos:

- *SchedulingOutput*, no caso direto;
- *IterativeSchedulingOutput*, no caso iterativo.

Em execuções experimentais, estes objetos são ainda serializados em ficheiros JSON, .txt, métricas e figuras.

Por fim, a validação: a solução produzida pelo *solver* é decodificada e verificada antes de ser convertida em métricas e ficheiros de saída.

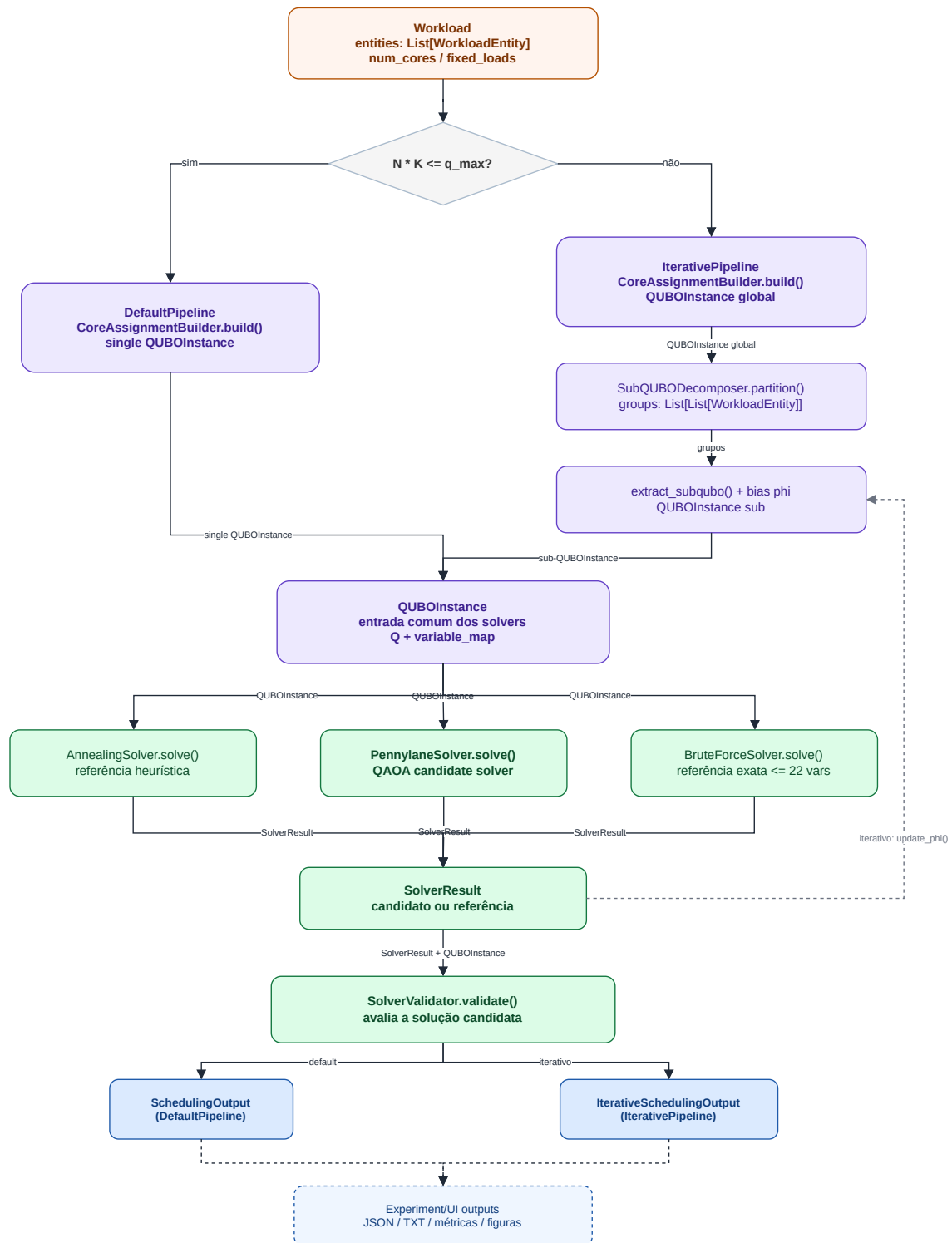


Figura 6.3: Fluxo final de construção, resolução e validação das QUBOInstance.

6.2 Estrutura do Projeto

A implementação foi organizada de forma a refletir as principais etapas da arquitetura. A diretoria `src/` contém o código principal do sistema, separado por módulos, enquanto os testes e cenários experimentais são mantidos em diretorias próprias.

Diretoria/Ficheiro	Função
<code>src/main.py</code>	Define <code>SchedulingEngine</code> e a execução por linha de comandos.
<code>src/data_contracts.py</code>	Centraliza as dataclasses e configurações partilhadas.
<code>src/builder/</code>	Constrói as matrizes QUBO.
<code>src/solver/</code>	Implementa os <i>solvers</i> e a validação das soluções.
<code>src/pipeline/</code>	Liga os módulos de construção, resolução e decomposição.
<code>src/decomposition/</code>	Agrupar snapshots reais e extrai <i>sub-QUBOs</i> .
<code>src/tracer/</code>	Recolhe processos reais do sistema.
<code>src/visualizer/</code>	Gera visualizações e tabelas de diagnóstico.
<code>src/experiments/</code>	Define e executa cenários experimentais.

Tabela 6.1: Mapa resumido da implementação.

6.2.1 Interfaces de Dados

As interfaces de dados definidas em `data_contracts.py` estabelecem a ligação entre os módulos da implementação. Cada etapa da arquitetura recebe uma estrutura de dados explícita, transforma-a e entrega uma nova interface à etapa seguinte. Esta escolha torna o fluxo mais previsível e evita que os módulos dependam diretamente dos detalhes internos uns dos outros.

Interface	Papel na implementação
<code>ProcessInfo</code>	Processo real capturado pelo tracer.
<code>SystemSnapshot</code>	Estado observado do sistema.
<code>WorkloadEntity</code>	Unidade abstrata a escalonar.
<code>Workload</code>	Problema de escalonamento entregue ao builder.
<code>ClusteredSnapshot</code>	Snapshot compactado após <i>clustering</i> .
<code>QUBOInstance</code>	Formulação QUBO entregue ao <i>solver</i> .
<code>SolverResult</code>	Resposta normalizada de um <i>solver</i> .
<code>SchedulingOutput</code>	Resultado final da pipeline direta.
<code>IterativeSchedulingOutput</code>	Resultado final da pipeline iterativa.

Tabela 6.2: Interfaces de dados centrais.

Como exemplo, a interface `WorkloadEntity` representa uma unidade abstrata a escalonar, independentemente de esta corresponder a um processo individual ou a um *bundle*. A sua definição reúne apenas os dados necessários às etapas seguintes:

```
@dataclass
class WorkloadEntity:
    entity_id: int
    cpu_weight: float
    rss_mb: float
```

```
label: str
```

Excerto de Código 6.1: Exemplo da interface de dados `WorkloadEntity`.

6.2.2 Configurações

Para além das interfaces de dados, a implementação separa os parâmetros de execução em objetos de configuração. Alguns módulos e algoritmos dependem de parâmetros ajustáveis, como limites de qubits, penalizações, critérios de decomposição entre outros.

Para manter a modularidade e reduzir o acoplamento entre componentes, estes parâmetros são agrupados em configurações explícitas. Desta forma, é possível alterar o comportamento da pipeline sem modificar a estrutura dos dados que circulam entre módulos.

Configuração	Papel na implementação
QUBOConfig	Define os parâmetros usados na construção da formulação QUBO.
QAOAConfig	Controla a execução do QAOA, incluindo camadas, passos, aprendizagem e tipo de mixer.
TracerConfig	Configura a recolha de processos reais pelo tracer.
DecompositorConfig	Define os limites e heurísticas usados na decomposição adaptativa.

Tabela 6.3: Configurações principais da implementação.

6.3 Conclusão

A arquitetura implementada foi pensada principalmente para reduzir acoplamento entre módulos. Como consequência, novas fontes de workload, novos *solvers* ou novas estratégias de decomposição podem ser introduzidos sem alterar a estrutura global da pipeline. Esta característica torna a plataforma adequada enquanto *testbed* para investigação e comparação de abordagens.

Investigação

Este capítulo descreve a metodologia adotada para a execução de experimentos e apresenta os resultados obtidos.

O objetivo em nenhum momento é demonstrar vantagem quântica, mas sim avaliar se o sistema produz soluções válidas, como se comporta sob diferentes configurações e onde se encontram os seus limites.

A execução dos experimentos foi assistida por um agente de inteligência artificial com acesso ao repositório, que implementou os cenários definidos, correu as simulações e gerou os relatórios de resultados. Todas as conclusões foram posteriormente revistas e validadas pelo autor. No total foram realizadas 1164 execuções principais, distribuídas por cinco fases experimentais.

7.1 Arquitetura dos Experimentos

Para avaliar o sistema em condições distintas, foi criada uma camada experimental em torno do *SchedulingEngine*. Esta camada separa a definição dos cenários da sua execução, o que permite variar sistematicamente a dimensão do problema, o número de processos, o limite de qubits, os parâmetros do QAOA e a estratégia de resolução sem alterar o código principal do sistema.

A arquitetura experimental organiza-se em quatro etapas: configuração do cenário, orquestração da execução, resolução pela pipeline selecionada e persistência dos resultados. A Figura 7.1 apresenta este fluxo. Os executores de cenários constituem a camada adicional sobre a arquitetura descrita no capítulo anterior; o restante processamento continua a ser realizado pelo *SchedulingEngine*.

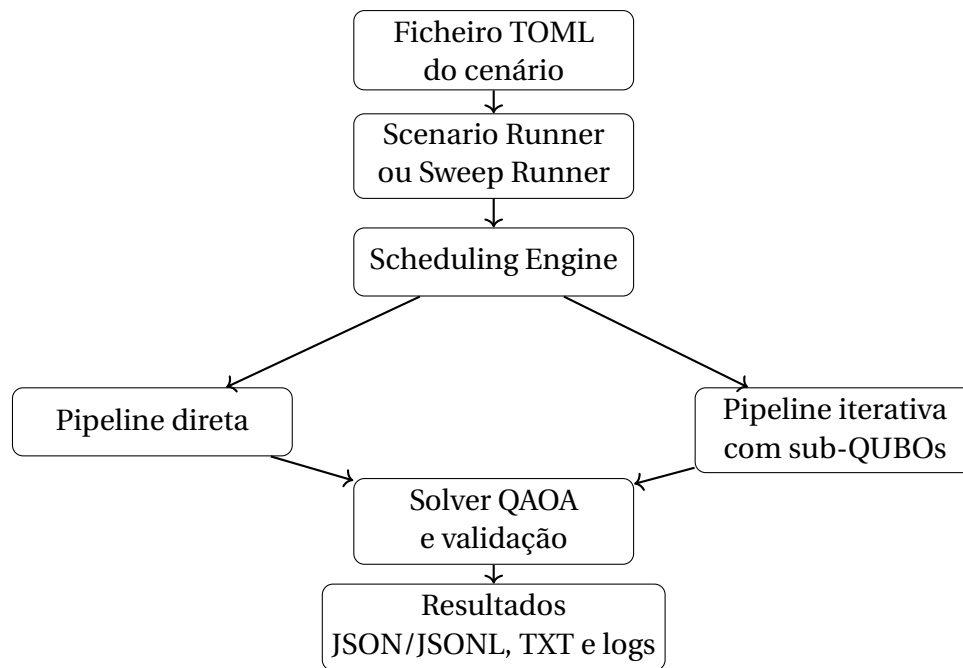


Figura 7.1: Arquitetura utilizada na execução dos experimentos da investigação.

7.1.1 Definição e Execução dos Cenários

Cada experimento é descrito num ficheiro de configuração em formato TOML. As secções do ficheiro definem o *workload*, a formulação QUBO, os parâmetros do QAOA, a decomposição, a validação e as opções de execução. Esta separação permite repetir uma experiência ou alterar uma única variável experimental sem modificar o código do sistema.

O *Scenario Runner* carrega a configuração, converte os seus valores nos objetos usados pelo sistema e executa o número de repetições especificado. Nos estudos paramétricos, o *Sweep Runner* expande os eixos e casos definidos no ficheiro TOML, gera as combinações correspondentes e entrega cada variante ao mesmo mecanismo de execução. Deste modo, tanto uma execução isolada como uma varredura de parâmetros seguem o mesmo percurso e produzem registos com a mesma estrutura.

Para cada variante, o *SchedulingEngine* seleciona a pipeline direta ou a pipeline iterativa. A primeira resolve a instância QUBO completa; a segunda decompõe o problema em sub-QUBOs quando o limite configurado de qubits é ultrapassado. A solução produzida pelo solver é depois validada e resumida num conjunto comum de métricas.

Um exemplo simplificado de um cenário experimental é apresentado de seguida:

```

scenario_version = 1
id = "T3.5"
name = "Three-core iterative decomposition"

[workload]
mode = "preset"
num_processes = 5
num_cores = 3
weights = [0.36, 0.27, 0.22, 0.18, 0.12]
  
```

```
[qubo]
penalty = 5.0

[qaoa]
layers = 1
steps = 50
learning_rate = 0.05
top_k = 20
mixer_type = "xy"

[decomposition]
qubit_max = 9
num_cores = 3
sorting_strategy = "WEIGHT_DESCENDING"

[execution]
pipeline = "iterative"
solvers = ["qaoa"]
repeats = 1
```

Excerto de Código 7.1: Exemplo simplificado de um cenário experimental em TOML.

7.1.2 Recolha e Armazenamento dos Resultados

Após cada execução, o sistema cria uma pasta própria para o cenário e guarda uma cópia da configuração utilizada. Cada repetição origina um ficheiro JSON com a configuração resolvida e as métricas, um ficheiro TXT legível e um log da saída produzida.

Em cada execução são registados, entre outros, os seguintes dados:

- configuração usada no experimento;
- número de processos, núcleos e variáveis QUBO;
- energia da solução obtida;
- viabilidade da solução;
- tempo de resolução;
- atribuição final de processos a cores;
- resultado da validação;
- desequilíbrio final de carga entre os núcleos;

7.2 Resultados da Investigação

7.2.1 Validação Inicial da Pipeline

Foram inicialmente executadas instâncias pequenas, com dois a quatro processos e dois cores. Estas instâncias produzem entre 4 e 8 variáveis QUBO, o que permite calcular o ótimo global por *brute force* e comparar diretamente a solução obtida pelo QAOA.

Processos	Cores	Variáveis	Viável	Ótimo	Pipeline
2	2	4	Sim	Sim	Direta
3	2	6	Sim	Sim	Direta
4	2	8	Sim	Sim	Direta

Tabela 7.1: Resultados dos testes iniciais em instâncias pequenas.

Em todos os casos a solução produzida foi viável e coincidiu com o ótimo obtido por *brute force*. Isto confirma que, para instâncias pequenas, o sistema executa corretamente o ciclo completo.

Numa instância com quatro processos e dois cores, foi também avaliado o efeito da profundidade p do circuito QAOA, ou seja, o número de vezes que os operadores de fase e de mistura são aplicados. Foram testados valores $p \in \{1, \dots, 7\}$ com cinco repetições cada. Em todos os casos $p = 1$ foi suficiente para atingir o ótimo, com o tempo a crescer de 1277 ms para 5111 ms sem qualquer melhoria de qualidade; esta instância era demasiado simples para discriminar o efeito da profundidade, cujo impacto real só se torna visível em instâncias maiores.

7.2.1.1 Validação da Decomposição

Para avaliar o comportamento da pipeline à medida que o número de processos cresce, foram executadas instâncias com $N \in \{2, 3, 4, 5, 6, 8, 10\}$ processos e dois cores, com $\text{qubit_max}=6$. Quando o número de variáveis QUBO excede este limite, o problema é automaticamente dividido em sub-QUBOs resolvidos pela pipeline iterativa.

N	Variáveis	Pipeline	Sub-QUBOs	Ótimo	Load imbalance	Tempo médio (ms)
2	4	Direta	0	100%	0.000	896.1
3	6	Direta	0	100%	0.333	1279.6
4	8	Iterativa	2	100%	0.000	2401.3
5	10	Iterativa	2	100%	0.200	2869.1
6	12	Iterativa	2	100%	0.000	3279.0
8	16	Iterativa	3	100%	0.000	4202.9
10	20	Iterativa	4	100%	0.000	5755.9

Tabela 7.2: Escalabilidade da pipeline com o número de processos.

A decomposição manteve a viabilidade e atingiu o ótimo global em todas as execuções até $N = 10$. O tempo cresceu com o número de sub-QUBOs, de 896 ms em $N = 2$ para 5756 ms em $N = 10$.

Os desequilíbrios de carga em $N = 3$ e $N = 5$ não resultam de falha da pipeline: com pesos uniformes e dois cores, é impossível distribuir um número ímpar de processos sem algum desequilíbrio.

7.2.1.2 Caracterização Inicial do QAOA

Foi utilizada uma instância fixa com oito processos e dois núcleos ($N = 8$, $K = 2$), correspondendo a 16 variáveis QUBO. Os pesos dos processos foram

$$\mathbf{w} = [0.30, 0.27, 0.22, 0.18, 0.15, 0.11, 0.08, 0.05],$$

O parâmetro `qubit_max` define o número máximo de variáveis que o QAOA pode resolver de uma vez; quando o problema excede esse limite, é dividido em sub-QUBOs resolvidos sequencialmente.

A profundidade p controla o número de camadas alternadas de operadores de fase e de mistura no circuito — maior profundidade aumenta a expressividade do circuito mas também o custo de simulação.

Os passos referem-se ao número de iterações do otimizador clássico que ajusta os parâmetros γ e β do circuito.

Cada resolução usa um circuito de profundidade $p = 2$ e 100 passos do otimizador clássico.

qubit_max	Pipeline	Sub-QUBOs	Gap médio	Tempo médio (ms)
4	Iterativa	4	0.000000	4550.3
6	Iterativa	3	0.000000	4546.8
8	Iterativa	2	0.000000	4777.0
10	Iterativa	2	0.000000	4792.5
12	Iterativa	2	0.000000	5648.0
16	Direta	0	0.000591	6911.1

Tabela 7.3: Comparação entre execução iterativa e direta para diferentes limites de qubits ($N = 8$, $K = 2$).

Nos casos em que o limite de qubits forçou a decomposição, a pipeline iterativa atingiu o ótimo global em todas as execuções. Com `qubit_max=16`, a pipeline resolveu a instância completa sem decomposição, mas não atingiu o ótimo com o orçamento de $p = 2$ e 100 passos, uma vez que o solver passou a otimizar um espaço de estados de maior dimensão com o mesmo orçamento algorítmico.

Este resultado sugere que a configuração $p = 2$ e 100 iterações não foi suficiente para explorar adequadamente o espaço de estados da instância. Para testar esta interpretação, manteve-se `qubit_max=16` e variaram-se a profundidade e o número de iterações do otimizador.

Profundidade p	Iterações	Gap	Ótimo	Tempo (ms)
2	100	0.000591	Não	8087.1
2	200	0.000591	Não	14141.7
2	500	0.001251	Não	35510.7
3	100	0.000000	Sim	10577.5
3	200	0.000000	Sim	20738.6
3	500	0.000000	Sim	51943.0
4	100	0.000000	Sim	13701.7
4	200	0.000000	Sim	27484.1
4	500	0.000000	Sim	68555.0

Tabela 7.4: Impacto da profundidade e do número de iterações na resolução direta com $\text{qubit_max}=16$ ($N = 8$, $K = 2$).

O aumento da profundidade de $p = 2$ para $p = 3$ permitiu recuperar o ótimo global com apenas 100 iterações. Em contraste, aumentar exclusivamente o número de iterações, mantendo $p = 2$, não melhorou a solução. Aliás, com 500 iterações, o gap foi inclusivamente superior ao observado com 100 e 200 iterações.

Estes resultados indicam que, nesta instância, a profundidade do circuito teve maior influência na qualidade da solução do que o número de iterações do otimizador. A resolução direta conseguiu, portanto, atingir o ótimo global, mas exigiu uma profundidade superior e maior tempo computacional.

No entanto, a pipeline iterativa atingiu a mesma qualidade em menos tempo nas configurações testadas. Este resultado demonstra uma vantagem computacional para esta instância e mostra que o aumento do limite de qubits deve ser acompanhado por uma parametrização adequada do QAOA.

Caracterização do Otimizador Clássico. Foi também avaliado o efeito do número de iterações do otimizador clássico e da inicialização de γ/β . Testaram-se valores entre 50 e 500 iterações e um grid de 25 pares de inicialização; em todos os casos a solução foi viável e ótima. Aumentar de 50 para 500 iterações multiplicou o tempo por um fator de aproximadamente 10 sem qualquer melhoria de qualidade, o que reforça que estas instâncias eram demasiado simples para revelar o limite do otimizador.

Os resultados da Secção 7.2.1.2 mostram que a qualidade depende não só do orçamento do circuito, mas também da forma como a pipeline gere a informação entre sub-QUBOs. As duas secções seguintes caracterizam os parâmetros que controlam essa gestão.

7.3 Escolha do Mixer

O mixer X aplica rotações independentes a cada qubit e não preserva a restrição de atribuição única durante a evolução do circuito, dependendo da penalidade P para desencorajar estados inválidos. O mixer XY atua apenas entre qubits do mesmo processo, preservando essa restrição estruturalmente e eliminando essa dependência.

O valor de P_{safe} é calculado pela heurística

$$P_{\text{safe}} \approx 2 \cdot w_{\text{max}} \cdot \frac{W_{\text{total}}}{K},$$

que estima a penalidade mínima necessária para que o custo de violar a restrição de atribuição única supere o ganho de qualquer atribuição inválida.

Foram totalizadas 504 execuções com duas instâncias. Na instância regular, usou-se $\mathbf{w} = [0.15, 0.35, 0.25, 0.25]$ e $P_{\text{safe}} = 0.35$. Na instância com processo dominante, usou-se $\mathbf{w} = [0.70, 0.10, 0.10, 0.10]$ e $P_{\text{safe}} = 0.70$.

7.3.1 Viabilidade sob orçamento reduzido

Primeiro foi fixado $p = 1$, $\text{top_k} = 1$, e o número de passos do otimizador entre 1 e 50. Os resultados agregam duas instâncias, duas penalidades e cinco sementes aleatórias para inicialização de γ e β , com 20 execuções por mixer em cada orçamento.

Passos	X viável	XY viável	X ótimo	XY ótimo
1	6/20	20/20	6/20	0/20
3	12/20	20/20	12/20	0/20
5	12/20	20/20	12/20	0/20
10	20/20	20/20	20/20	0/20
20	20/20	20/20	19/20	0/20
50	20/20	20/20	17/20	0/20

Tabela 7.5: Viabilidade e otimalidade com $\text{top_k} = 1$ e $p = 1$

O mixer XY manteve viabilidade em 120/120 execuções e atribuiu sempre 100% da massa de probabilidade ao subespaço de atribuições únicas válidas. O mixer X falhou a restrição de atribuição única em 30/120 execuções, com todas as falhas abaixo de 10 passos.

Contudo, o padrão de otimalidade inverteu-se: com $\text{top_k} = 1$, o mixer X produziu 86/120 soluções ótimas, enquanto o mixer XY produziu 0/120. A razão é estrutural: o XY parte de um estado inicial onde todos os processos estão atribuídos ao núcleo 0, e com circuitos pouco profundos a distribuição permanece concentrada em atribuições viáveis mas subótimas. O estado ótimo global foi classificado em primeiro lugar pelo mixer X nas execuções, mas apareceu nas posições 6–10 para o mixer XY na instância de referência.

7.3.2 Comparação com Valores Práticos de top_k

Para complementar o experimento anterior em que foi usado $\text{top_k}=1$, foi realizado um segundo conjunto de experimentos com $p \in \{2, 3\}$, 100 passos, P_{safe} e $\text{top_k} \in \{2, 3, 5, 10\}$. Todas as 160 execuções foram viáveis.

Instância	p	top_k	X ótimo	XY ótimo
Regular	2	2	5/5	0/5
Regular	2	3	5/5	0/5
Regular	2	5	5/5	0/5
Regular	2	10	5/5	5/5
Regular	3	2	5/5	1/5
Regular	3	3	5/5	2/5
Regular	3	5	5/5	2/5
Regular	3	10	5/5	5/5
Dominante	2	2	5/5	2/5
Dominante	2	3	5/5	2/5
Dominante	2	5	5/5	5/5
Dominante	2	10	5/5	5/5
Dominante	3	2	5/5	2/5
Dominante	3	3	5/5	2/5
Dominante	3	5	5/5	5/5
Dominante	3	10	5/5	5/5

Tabela 7.6: Otimalidade por mixer, profundidade e top_k com 100 passos e P_{safe} .

O mixer X atingiu o ótimo em 80/80 execuções. O mixer XY atingiu 43/80, com desempenho a melhorar consistentemente com top_k : ambos convergiram para 100% de otimalidade em $\text{top_k}=10$. O tempo médio foi 2787.0 ms para X e 2200.7 ms para XY, uma redução de 21%.

Este resultado clarifica o compromisso observado anteriormente: a dificuldade do XY em optimalidade não é absoluta, mas depende do número de candidatos disponíveis. Com $\text{top_k}=10$, a distribuição do XY contém candidatos suficientes para capturar o estado ótimo; com valores menores, a concentração de massa em atribuições viáveis mas subótimas limita a recuperação.

7.3.3 Síntese

Os experimentos revelam um compromisso claro entre os dois mixers. O XY é estruturalmente superior em viabilidade: manteve 100% de massa no subespaço de atribuições únicas válidas em todas as execuções e nunca produziu uma solução inválida, independentemente da penalidade ou do orçamento do otimizador. Foi também 16–22% mais rápido em todos os regimes testados.

Em optimalidade, o comportamento depende do número de candidatos disponíveis. Com $\text{top_k}=1$, o mixer X superou claramente o XY — 86/120 ótimos contra 0/120 — porque o estado ótimo global tende a aparecer em posições mais elevadas na sua distribuição. Com valores práticos de top_k , essa vantagem diminui: a partir de $\text{top_k}=10$, ambos os mixers atingiram 100% de otimalidade.

A escolha do mixer deve portanto considerar o regime de operação. O XY é preferível quando a viabilidade é prioritária ou quando top_k é suficientemente grande; o mixer X é mais robusto em optimalidade quando o orçamento de candidatos é muito restrito, desde que o otimizador tenha passos suficientes para garantir viabilidade.

7.4 Limites da Pipeline

As fases anteriores caracterizaram o comportamento da pipeline em condições favoráveis: workloads uniformes e instâncias pequenas. Nessas condições, a pipeline atingiu o ótimo na quase totalidade das execuções. Para identificar onde e como a qualidade se degrada, foram construídas instâncias deliberadamente difíceis e alargado o espaço de parâmetros investigado.

7.4.1 Instâncias com Processo Dominante

A primeira estrutura consiste em seis processos onde um tem peso muito superior aos restantes. Nestes casos, a atribuição ótima é estruturalmente forçada: o processo dominante deve ir para um núcleo e os cinco leves para o outro; qualquer outra distribuição é fortemente penalizada. Foram testados pesos dominantes em $\{0.60, 0.70, 0.80, 0.90, 0.95\}$, com os restantes uniformes, e combinações $p \in \{1, 2\}$ e $\text{top_k} \in \{3, 10\}$.

p	top_k	Ótimos	Gap médio	Tempo médio (ms)
1	3	24/25	0,006755%	2003,0
1	10	25/25	0	1999,3
2	3	9/25	0,082395%	3004,6
2	10	25/25	0	3000,1

Tabela 7.7: Resultados com processo dominante ($N = 6$, $K = 2$).

O resultado mais relevante é que aumentar p de 1 para 2 degradou fortemente a solução quando $\text{top_k} = 3$: de 24/25 para 9/25 ótimos.

Com $\text{top_k} = 10$, a profundidade deixou de fazer diferença e ambas as configurações atingiram o ótimo em todas as repetições.

Este padrão revela que maior profundidade não implica necessariamente melhor qualidade — em circuitos mais profundos, a distribuição de probabilidade pode dispersar-se por mais estados, exigindo um conjunto de candidatos maior para capturar o ótimo.

7.4.2 Instâncias com Pesos Quase Iguais

A segunda estrutura adversarial usa quatro processos com pesos próximos entre si, onde o desequilíbrio máximo entre o processo mais leve e o mais pesado varia entre 0,01 e 0,20. Estas instâncias são difíceis porque o ótimo global tem energia muito semelhante a soluções subótimas, tornando a distinção difícil para o otimizador.

p	top_k	Ótimos	Gap médio	Tempo médio (ms)
1	3	15/20	0,003213%	1569,5
1	10	20/20	0	1566,0
2	3	8/20	0,005712%	2167,5
2	10	20/20	0	2167,1

Tabela 7.8: Resultados com pesos quase iguais ($N = 4$, $K = 2$).

O padrão é consistente com o anterior: top_k = 10 eliminou todas as falhas, enquanto top_k = 3 deixou uma fração de runs subótimos. Os gaps são pequenos em valor absoluto, mas não equivalem a optimalidade exata. O simulated annealing atingiu o ótimo em 80/80 execuções neste conjunto.

7.4.3 Instâncias com Estrutura CPU/I/O

A terceira estrutura adversarial foi concebida para testar a decomposição adaptativa. Foram criados seis processos divididos em dois grupos distintos: três CPU-bound (pesos altos, I/O wait baixo) e três I/O-bound (pesos baixos, I/O wait alto).

O parâmetro io_alpha controla o peso relativo do perfil de I/O na afinidade da decomposição.

io_alpha	Ótimos	Coupling intra	Coupling inter	Load imbalance	Tempo (ms)
0,1	20/20	0,977	0,295	0,561	716,3
0,5	20/20	0,984	0,294	0,647	715,5
0,9	20/20	0,987	0,293	0,732	714,0

Tabela 7.9: Resultados do clustering CPU/I/O para diferentes valores de io_alpha ($N = 6$, $K = 2$).

Em todas as configurações, a decomposição separou corretamente os três processos CPU-bound dos três I/O-bound. As 60 execuções atingiram o ótimo.

No entanto, aumentar io_alpha aumentou o desequilíbrio de carga: ao privilegiar a afinidade de I/O na formação dos clusters, os bundles ficam com pesos totais mais desiguais. Este resultado mostra que afinidade da decomposição e qualidade de scheduling são objetivos potencialmente conflituosos — uma decomposição mais pura em termos de perfil de I/O não implica necessariamente uma atribuição mais equilibrada.

Conclusões e Trabalho Futuro

8.1 Conclusões Principais

Este projeto teve como objetivo construir e validar uma plataforma híbrida quântico-clássica completa para o problema de escalonamento de processos, isolando a decisão de atribuição processo-núcleo e expressando-a numa formulação compatível com otimizadores quânticos. O resultado central deste trabalho é a própria plataforma: uma pipeline funcional, modular e validada, capaz de transformar um snapshot de processos numa solução de escalonamento através de QAOA.

A investigação conduzida sobre esta plataforma serviu sobretudo para confirmar que a pipeline se comporta como esperado: nas instâncias pequenas onde o ótimo pode ser calculado exatamente, a solução produzida foi sempre viável e ótima; à medida que o número de processos cresceu, a pipeline iterativa manteve essa qualidade recorrendo à decomposição em sub-QUBOs; e mesmo em instâncias construídas deliberadamente para serem difíceis, a plataforma manteve-se estável, desde que os parâmetros do QAOA fossem ajustados de forma coerente entre si.

Este trabalho demonstra que é possível construir uma plataforma híbrida funcional para escalonamento de processos preparada para ser benéfica com melhorias futuras no hardware quântico.

8.2 Trabalho Futuro

Uma limitação explícita deste trabalho é que toda a execução foi realizada em simulação, através dos backends `lightning.gpu` e `lightning.qubit` do PennyLane, e não em hardware quântico real. A validação em dispositivos físicos permitiria caracterizar o impacto do ruído e da conectividade limitada entre qubits físicos na qualidade das soluções obtidas, algo que a simulação não pode captar.

Adicionalmente, a plataforma foi validada sobretudo em instâncias de pequena e média dimensão. Explorar sistematicamente instâncias maiores permitiria caracterizar com mais detalhe o comportamento da pipeline iterativa ao longo de um número elevado de sub-QUBOs sequenciais, nomeadamente até que ponto a propagação de carga através do vetor ϕ preserva a qualidade da solução global à medida que a cadeia de decomposição se alonga.

Bibliografia

- [1] The Linux Kernel Development Community. CFS scheduler. <https://docs.kernel.org/scheduler/sched-design-CFS.html>. Accessed: 2026-06-18.
- [2] Edward Farhi, Jeffrey Goldstone, and Sam Gutmann. A quantum approximate optimization algorithm, 2014.
- [3] Gary Kochenberger, Jin-Kao Hao, Fred Glover, Mark Lewis, Zhipeng Lü, Haibo Wang, and Yang Wang. The unconstrained binary quadratic programming problem: A survey. *Journal of Combinatorial Optimization*, 28(1):58–81, 2014.
- [4] Fred Glover, Gary Kochenberger, and Yu Du. A tutorial on formulating and using QUBO models, 2018.
- [5] Ville Bergholm, Josh Izaac, Maria Schuld, Christian Gogolin, Shahnawaz Ahmed, Vishnu Ajith, M. Sohaib Alam, Guillermo Alonso-Linaje, B. AkashNarayanan, Ali Asadi, et al. PennyLane: Automatic differentiation of hybrid quantum-classical computations, 2018.
- [6] Marco Cerezo, Andrew Arrasmith, Ryan Babbush, Simon C. Benjamin, Suguru Endo, Keisuke Fujii, Jarrod R. McClean, Kosuke Mitarai, Xiao Yuan, Lukasz Cincio, and Patrick J. Coles. Variational quantum algorithms. *Nature Reviews Physics*, 3:625–644, 2021.
- [7] Ali Asadi et al. Hybrid quantum programming with PennyLane Lightning on HPC platforms, 2024.
- [8] Andrew Lucas. Ising formulations of many NP problems. *Frontiers in Physics*, 2:5, 2014.
- [9] Stuart Hadfield, Zhihui Wang, Bryan O’Gorman, Eleanor G. Rieffel, Davide Venturelli, and Rupak Biswas. From the quantum approximate optimization algorithm to a quantum alternating operator ansatz, 2017.
- [10] Zhihui Wang, Nicholas C. Rubin, Jason M. Dominy, and Eleanor G. Rieffel. XY-mixers: Analytical and numerical results for QAOA, 2019.
- [11] Franz G. Fuchs, Kjetil Olsen Lye, Halvor Møll Nilsen, Alexander J. Stasik, and Giorgio Sartor. Constrained mixers for the quantum approximate optimization algorithm, 2022.
- [12] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization, 2014.
- [13] John Preskill. Quantum computing in the NISQ era and beyond. *Quantum*, 2:79, 2018.

-
- [14] Andrew Y. Ng, Michael I. Jordan, and Yair Weiss. On spectral clustering: Analysis and an algorithm. In *Advances in Neural Information Processing Systems*, volume 14, pages 849–856. MIT Press, 2001.